

TESS Backend-Only Authentication

1. Overview

1.1 Purpose

This document defines the backend-only authentication architecture for the Proxy API.

Goals:

- **Single session mechanism**
One HttpOnly cookie (`accessToken`) is the only session store.
- **Backend-owned authentication flows**
SSO (Microsoft Entra), Tenant (Entra CIAM), and Punchout flows are fully implemented on the backend.
- **Frontend simplification**
The frontend only:
 1. Starts the authentication flow (redirect or one-time POST)
 2. Sends API requests using the cookie
- **Eliminate race conditions**
No token syncing between frontend and backend.
No dual profile hooks.
No manual cookie handling in JavaScript.

1.2 Out of Scope for Frontend (**I already implemented this on my FE branch**)

The frontend must **not**:

- Implement OAuth code exchange
- Store or refresh tokens
- Handle identity provider callbacks
- Store long-lived credentials

The frontend **only**:

- Redirects the browser to login endpoints
 - Sends requests with `credentials: 'include'`
-

2. Architecture

2.1 High-Level Flow

SSO:

Frontend → GET /auth/sso → Microsoft Entra
→ GET /auth/sso/callback → Backend sets cookie → Redirect to app

Tenant:

Frontend → GET /auth/tenant → Entra CIAM
→ GET /auth/tenant/callback → Backend sets cookie → Redirect to app

Punchout:

Frontend → POST /auth/punchout/consume
→ Backend sets cookie → 200 OK

After any flow:

All API calls use the same cookie session

2.2 Session Cookie

Attribute	Value
Name	<code>accessToken</code>
Type	JWT (signed by backend)
HttpOnly	Yes
Secure	Yes
SameSite	<code>None</code>
Path	<code>/</code>
Max-Age (SSO / Tenant)	24 hours
Max-Age (Punchout)	12 hours

Payload (SSO/Tenant):

```
{ userId: number }
```

Payload (Punchout):

```
{ userId: number, punchout: true }
```

All protected routes use the same middleware to read:

```
req.user.userId  
req.user.punchout (optional)
```

3. Authentication Flows

3.1 SSO (Microsoft Entra – Common)

Use case: Internal or B2B users authenticate using Microsoft Entra (common tenant).

Step	Actor	Action
1	Frontend	Redirect user to <code>GET /auth/sso</code>
2	Backend	Redirect to Microsoft Entra authorize endpoint
3	Microsoft	Redirect browser to <code>/auth/sso/callback?code=...</code>
4	Backend	Exchange code for tokens, fetch user profile, create DB user, issue session cookie
5	Frontend	User lands in app; all API calls use cookie

Frontend behavior:

- Trigger redirect to `/auth/sso`
 - Do not handle tokens or callback parameters
-

3.2 Tenant (Microsoft Entra CIAM)

Use case: External or customer users authenticate via CIAM tenant.

Step	Actor	Action
1	Frontend	Redirect user to <code>GET /auth/tenant</code>
2	Backend	Redirect to CIAM authorize endpoint
3	CIAM	Redirect browser to <code>/auth/tenant/callback?code=...</code>
4	Backend	Exchange code, fetch userinfo, create DB user, issue session cookie
5	Frontend	User lands in app; all API calls use cookie

Frontend behavior:

- Same pattern as SSO
 - Only URL differs
-

3.3 Punchout

Use case: User enters from external procurement system with a punchout token.

Step	Actor	Action
1	Frontend	Call <code>POST /auth/punchout/consume</code> with token
2	Backend	Verify token, issue session cookie
3	Frontend	Use app normally with cookie-based API calls

Frontend behavior:

- Send token once
 - Do not store the punchout token
 - All future requests rely on cookie
-

4. API Reference

Method	Path	Purpose
GET	/auth/sso	Start SSO flow
GET	/auth/sso/callback	OAuth callback
GET	/auth/tenant	Start tenant flow
GET	/auth/tenant/callback	OAuth callback
POST	/auth/punchout/consume	Consume punchout token

Swagger UI:

[/swagger](#)

Swagger file:

[src/features/auth/authSwagger.ts](#)

5. Configuration

5.1 Environment Variables

Variable	Description	Example
APP_URL	Frontend redirect after login	https://tessix.no
API_BASE_URL	Backend base URL	https://api.tessix.no
AUTH_MICROSOFT_ENTRA_ID_ID_SSO_USER	Entra client ID	UUID

<code>AUTH_MICROSOFT_ENTRA_ID_SECRET_SSO_USER</code>	Entra secret	Secret
<code>AUTH_MICROSOFT_ENTRA_ID_ISSUER_SSO_USER</code>	Entra issuer	<code>https://login.microsoftonline.com/common/v2.0</code>
<code>AUTH_MICROSOFT_ENTRA_ID_ID_TENANT_USER</code>	CIAM client ID	UUID
<code>AUTH_MICROSOFT_ENTRA_ID_SECRET_TENANT_USER</code>	CIAM secret	Secret
<code>AUTH_MICROSOFT_ENTRA_ID_ISSUER_TENANT_USER</code>	CIAM issuer	<code>https://<tenant>.ciamlogin.com/...</code>

Key Vault is used in production.
`process.env` is used locally.

6. Error Handling

6.1 SSO / Tenant Callback Errors

Redirect to:

`{APP_URL}?auth_error=<code>`

Code	Meaning
<code>auth_failed</code>	Token exchange or profile fetch failed
<code>missing_code</code>	No OAuth code provided
<code>no_token</code>	Token response invalid
<code>not_configured</code>	Flow is not configured

Frontend should:

- Display error
- Optionally redirect back to login

6.2 Punchout Errors

HTTP Status	Meaning
200	Success
401	Missing token
403	Invalid or expired token
500	Server error

Response:

```
{ "message": "... " }
```

6.3 Protected Routes

If cookie is missing or invalid:

- Backend returns 401 or 403
- Frontend redirects to login endpoint

7. Security

- OAuth tokens never reach frontend
- HttpOnly cookie prevents JavaScript access
- Secure + SameSite=None allows cross-origin API calls
- Punchout token is one-time use only
- Backend verifies all JWTs

8. Frontend Integration Checklist

SSO / Tenant

- Redirect to `/auth/sso` or `/auth/tenant`
- Do not parse OAuth codes
- Use `credentials: 'include'` for all API calls
- Handle `?auth_error=...` if present

Punchout

- Read token
- Call `/auth/punchout/consume` once
- Do not store token
- Continue using cookie-based API

General

- No tokens in localStorage or memory
 - On 401/403 → redirect to login
 - All requests go through same API base URL
-

9. Swagger

- UI: `/swagger`
- Tag: `Auth`
- File: `src/features/auth/authSwagger.ts`

